

A Shrunk Logistic Predictor for Cold Start Benchmarks

Technical report for the NeurIPS 2026 Predictive AI Evaluation Competition, Codabench user aghasalim. Code: <https://github.com/aghasalim/predictive-eval>

Summary

The task is to predict whether an AI system answers a benchmark item correctly, on a benchmark never seen in training, while labels for that system and benchmark are revealed a few at a time. My predictor is a small logistic model with four parts: a prior ability for each system, learned from the training benchmarks by model name; a difficulty for the new benchmark, estimated online; an offset for each system on that benchmark; and online offsets for items that share a tag or are the same question. Every online part is a MAP estimate under a Gaussian prior, so one early label moves a prediction a little rather than to 0 or 1.

On a local copy of the scoring protocol, with one training benchmark held out at a time, the final model (v3) scores 0.2146 Brier ALC against 0.2625 for the empirical mean baseline and 0.2500 for a constant 0.5. Most of the gain is careful shrinkage of the online estimates; ability learned on other benchmarks barely helps before the first label. My best platform submission scored 0.179449. Two ideas that use the item text, cheap text features and sentence embeddings of the question, gave no gain on held out benchmarks, and I report both.

1. Task and data

Each prediction is for a (subject, item) pair, where a subject is one AI system under one harness and an item is one benchmark question. The predictor sees the subject's visible attributes, the item's text, its tag string and an anonymous benchmark id, and a list of labels revealed so far. For every subject and benchmark pair, labels arrive in a random order at budgets of 0, 1, 3, 7, 15 and 31 per pair, and the score is the Brier score on held out items at each budget, combined as

$$\text{ALC} = 0.1 B_0 + 0.2 B_1 + 0.2 B_3 + 0.2 B_7 + 0.2 B_{15} + 0.1 B_{31}.$$

Lower is better, and predicting 0.5 everywhere scores 0.25.

The only training data is the public release of measurement-db (aims-foundations/measurement-db on Hugging Face). It has six benchmarks. I used the five with right or wrong responses and left out mmdocrag, whose responses are fractions.

benchmark	subjects	items	responses	pairs with 80+ items	accuracy
matharena	104	1,751	92,861	81	65%
multi_swebench	82	2,126	57,975	82	15%
real_webagents	33	233	4,449	26	36%
researchcodebench	31	212	6,784	31	36%

benchmark	subjects	items	responses	pairs with 80+ items	accuracy
swe_rebench	1	6,306	67,074	1	48%

swe_rebench has a single subject, so on it the model can only learn item effects, and the subject offsets have nothing to share.

2. Local evaluation protocol

Each submission to the platform is scored on a freshly drawn sample of about 9 pairs, so one platform score is noisy (Section 5). To compare versions I rebuilt the scoring protocol locally in `sim.py`, as close to the published rules as I could read them:

1. Hold out one of the five benchmarks and fit on the other four.
2. Take every subject with at least 80 distinct items on the held out benchmark. In each round, sample up to 8 of them.
3. For each sampled pair, shuffle its items and split them in half: an acquisition pool and an evaluation pool (capped at 60 items).
4. At each budget, reveal the first n acquisition labels of every pair in the round, predict every evaluation item, and record the Brier score.
5. Combine the budgets with the ALC weights and average over pairs.

The held out benchmark is given an anonymous id, as on the platform. A cell is one held out benchmark and one random seed, with 8 rounds per cell, and all the numbers in this report use the same 20 cells (5 benchmarks by 4 seeds) for every model. Comparisons between versions are paired over those 20 cells, so the noise from the draw is the same for both sides.

3. Model

For subject s , item i on benchmark b , the final model predicts

$$P(\text{correct}) = \text{sigmoid}(a_b + w \theta_s + d_{sb} + g_b(\text{tag}_i) + b_i).$$

θ_s , ability by name. On each training benchmark I take each subject's smoothed accuracy, $(\text{correct} + 2m) / (n + 2)$ with m the benchmark's mean, convert it to a logit, and subtract the benchmark's mean logit. A model name's ability is the average of these over the benchmarks where it appears, shrunk towards 0 by one pseudo benchmark. A name never seen in training gets 0. The weight w is 0.5.

a_b , benchmark difficulty. At budget 0 this is the average benchmark logit from training. Once labels arrive, it is the MAP estimate from every revealed label on the benchmark, with a Gaussian prior of standard deviation 0.7 around that average.

d_{sb} , the subject's own offset. The MAP estimate from that subject's revealed labels with the other terms fixed, prior standard deviation 1.0. It stays near 0 until the subject has labels of its own.

$g_b(\text{tag})$, tag offset (added in v2). Items on some benchmarks carry a tag string, for example a programming language. Items sharing a tag get a common MAP offset from all revealed labels with that

tag, prior standard deviation 1.0.

b_i, question offset (added in v3). Each pair splits its items independently, so a question being predicted for one subject has often already been revealed for another. At budget 31, between 13% and 72% of evaluation items had a label from another subject on four of the five benchmarks (none on swe_rebench, which has one subject). The question's own offset is the MAP estimate from every revealed label on that exact question, keyed by benchmark id and question text, prior standard deviation 2.0.

All MAP estimates use 8 Newton steps on the one dimensional posterior, fitted in the order a_b, g, d, b_i, each with the earlier terms fixed. The code is pure Python apart from pandas and numpy in fitting, and the estimates for a given set of labels are computed once and cached.

4. Local results

Brier ALC on the 20 shared cells, lower is better.

model	ALC	matharena	multi_swebench	real_webagents	researchcodebench	swe_rebench
constant	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500
0.5						
empirical mean	0.2625	0.2705	0.2036	0.2402	0.2691	0.3289
v1	0.2203	0.2359	0.1702	0.2004	0.2196	0.2753
v2	0.2171	0.2323	0.1720	0.1974	0.2085	0.2753
v2 with text features	0.2242	0.2366	0.1737	0.1956	0.2240	0.2910
v3	0.2146	0.2279	0.1705	0.1950	0.2043	0.2753
v4	0.2143	0.2263	0.1700	0.1949	0.2047	0.2757

Paired differences over the 20 cells (negative means the first model is better):

comparison	difference	standard error	first better	first worse
v1 against empirical mean	-0.0422	0.0021	20 of 20	0 of 20
v2 against v1	-0.0032	0.0010	12 of 20	4 of 20
v2 with text features against v2	+0.0071	0.0017	4 of 20	16 of 20
v3 against v2	-0.0025	0.0004	16 of 20	0 of 20
v4 against v3	-0.0003	0.0002	12 of 20	8 of 20

v1 improves on the empirical mean by 0.0422 and on the constant by 0.0297, and the budget table below shows where that comes from. At budget 0, where only the ability prior and the training average are available, v1 scores 0.2508, no better than a constant: ability learned on other benchmarks barely transfers to an unseen one. The gain is shrinkage once labels arrive. After one label the empirical mean predicts 0 or 1 and scores 0.3912, while v1 moves a little and scores 0.2331. The tag offsets (v2) and the question offsets (v3) then each add a small gain that holds in most cells and grows with the budget, since both need revealed labels.

Mean Brier score by label budget, averaged over all cells:

model	0	1	3	7	15	31
empirical mean	0.2500	0.3912	0.2617	0.2237	0.2099	0.2016
v1	0.2508	0.2331	0.2225	0.2132	0.2068	0.2008
v2	0.2508	0.2341	0.2216	0.2097	0.1996	0.1903
v2 with text features	0.2675	0.2454	0.2288	0.2141	0.2025	0.1927
v3	0.2508	0.2334	0.2203	0.2076	0.1954	0.1817
v4	0.2508	0.2341	0.2204	0.2071	0.1943	0.1806

5. Platform results

version	submissions	platform ALC
v1	953625, 953903, 953905, 953907	0.193979, 0.193899, 0.191791, 0.198023
v2	953862, 953902, 953904, 953906, 953908	0.213320, 0.173103, 0.179449, 0.163046, 0.188681
v3	954619	0.182867

Each submission is scored on a new sample, so I submitted v1 and v2 several times, alternating them. The means are 0.1944 for v1 and 0.1835 for v2, a difference of 0.0109 in v2's favour with a standard error of 0.0086. Subtracting each submission's own budget 0 score, which is the same model for both versions and so measures how hard the sample was, gives 0.0094 with a standard error of 0.0075. Both agree in direction with the local comparison but are only a little over one standard error. v3's single score lies inside v2's range, as expected for a change of 0.0025 locally. The leaderboard entry is 953904 (v2, 0.179449).

The platform scores are lower than the local ones. The platform's benchmarks and subjects differ from measurement-db, and I cannot tell which part of the gap comes from that.

6. What did not work

Text features (v2 ablation). A ridge regression from eight cheap descriptors of the item text (length, line count, digit and maths symbol share, code markers, URLs, error words) to how much harder an item is than its benchmark's average. It fits the training benchmarks but does not transfer: switched on, it costs 0.0071 on held out benchmarks. I kept it in the code with weight 0.

Sentence embedding neighbours (v4). Every revealed label leaves a residual against the model without item offsets, and a question takes a similarity weighted average of other questions' residuals on the same benchmark, with similarity from all-MiniLM-L6-v2 vectors. I first checked that the scoring container can load sentence-transformers with a submission that only imported it (954620, which predicted 0.5 and scored 0.250000). Over 11 settings of strength, width and shrinkage, the best changed the score by -0.0003 (standard error 0.0002), which is nothing once the choice among 11 is counted. It helps a little on matharena and hurts on researchcodebench, whose items are long and open with shared text, so unrelated questions looked alike. Centring the vectors on each benchmark's mean and

embedding both ends of long items removed most of that damage but did not leave a gain. The tuning logs are `tune4.log` and `tune4b.log`.

7. Limitations

- The prior settings (w, the prior standard deviations) were chosen on the same local protocol that reports the results, from small grids (`tune.py`). The local numbers are therefore slightly optimistic for every learned version, though the paired differences between versions are less affected, since each version only adds one setting on top of the last.
- The local protocol is my reading of the rules. The platform's acquisition order, pair sampling and evaluation pool sizes may differ.
- Ability is matched by exact model name. A system whose name never appears in training gets the average, and versions of the same model under different names share nothing.
- Five benchmarks give five held out folds. Any claim about transfer to a new kind of benchmark rests on those five.
- The question offset needs other subjects on the same benchmark in the same sample. If the platform scores pairs separately, v3 reduces to v2.

8. Reproducing

Code: <https://github.com/aghasalim/predictive-eval>

```
python -m venv .venv && .venv/bin/pip install pandas pyarrow numpy sentence-transformers
./fetch.sh                                     # measurement-db (gated) and the organisers'
                                              baselines
.venv/bin/python report/data_summary.py        # Section 1 table
.venv/bin/python report/local_scores.py        # Section 4 tables, about 20 minutes on a laptop CPU
.venv/bin/python report/check_report.py        # every number in this report, recomputed
.venv/bin/python repeats.py                    # Section 5 comparison
```

The submitted archives are built from `submission/`, `submission2/` and `submission3/`, each a `model.py` with its fitted parameters in JSON. The empirical mean baseline is the organisers' `empirical_mean/model.py` from `aims-foundations/paiec_baseline`.

9. Disclosures

Training data: only the public measurement-db release. The v4 experiment used the pretrained all-MiniLM-L6-v2 sentence encoder, which was not fine tuned. I used an LLM coding assistant while writing the code and this report; the method, the experiments and the numbers were checked by running the scripts listed above.